

MOVE HARD SPORTS · 2026

Opis wszystkich testów

Szczegółowy opis każdego testu jednostkowego i E2E w projekcie MHS

124TESTÓW
ŁĄCZNIE**81**

JEDNOSTKOWYCH

43

E2E

12PLIKI
TESTOWE

Vitest + Testing Library (jednostkowe) · Playwright (E2E)

Spis treści

12 plików testowych — kliknij numer strony aby przejść do sekcji

TESTY JEDNOSTKOWE (VITEST)

1.	Header	header.test.tsx	10
2.	DisciplinePicker	discipline-picker.test.tsx	17
3.	Strona logowania	login/page.test.tsx	8
4.	Strona rejestracji	register/page.test.tsx	12
5.	Formularz onboarding	onboarding-form.test.tsx	12
6.	Formularz kampanii	campaign-form.test.tsx	16
7.	Przycisk usuwania	delete-button.test.tsx	6

TESTY E2E (PLAYWRIGHT)

8.	Rejestracja	registration.spec.ts	5
9.	Logowanie i wylogowanie	login.spec.ts	7
10.	Onboarding	onboarding.spec.ts	8
11.	Dashboard — kontrola dostępu	dashboard.spec.ts	4
12.	Kampanie (lista, tworzenie, podgląd, edycja, usuwanie)	campaigns.spec.ts	19

1

Header

`src/components/header/header.test.tsx`

JEDNOSTKOWY

10 TESTÓW

GDY UŻYTKOWNIK NIE JEST ZALOGOWANY

T1 renders navigation links

Renderuje nagłówek z nawigacją publiczną (`publicNav` zawierający "For Athletes" i "For Brands") i sprawdza, że oba linki pojawiają się w DOM. Używa `getAllByText() [0]`, bo header może renderować te elementy kilka razy (wersja desktop + mobile).

T2 renders sign in link pointing to /login

Sprawdza, że istnieje link z rolą `link` i tekstem dopasowanym przez `/sign in/i` (bez rozróżnienia wielkości liter). Weryfikuje atrybut `href="/login"`. Link mockowany przez `next/link → <a>`.

T3 renders join link pointing to /register

Analogicznie do T2 — sprawdza link "Join" wskazujący na `/register`.

T4 renders logo image

Sprawdza obecność elementu `<img>` z `alt="Your Company"` — logo firmy w headerze.

T5 renders mobile menu button with screen reader label

Sprawdza obecność tekstu "Open main menu" — jest to etykieta screen readera (wizualnie ukryta przez klasę `sr-only`) wewnątrz przycisku hamburger menu mobilnego.

GDY UŻYTKOWNIK JEST ZALOGOWANY

T6 renders dashboard link

Przekazuje `user={{ email: 'test@example.com' }}`. Sprawdza, że istnieje link "Dashboard" z `href="/dashboard"`. Wyszukuje przez `/dashboard/i` (bez rozróżnienia wielkości liter).

T7 does not render sign in link

Sprawdza, że dla zalogowanego użytkownika link "Sign in" NIE istnieje w DOM — używa `queryByRole()` zamiast `getByRole()` i asercji `.not.toBeInTheDocument()`.

GDY SHOWSIGNOUT = TRUE

T8 renders navigation links

Renderuje z `navigation={dashboardNav}` i `showSignOut` (bez wartości = `true`). Sprawdza, że linki "My Campaigns" i "My Profile" są widoczne w DOM.

T9 renders sign out button

Sprawdza, że istnieje przycisk z rolą `button` i tekstem dopasowanym przez `/sign out/i`. W kodzie tekst to "SIGN OUT" (CSS `uppercase`), ale `getByRole()` ignoruje CSS i korzysta z treści DOM — test działa poprawnie.

T10 does not render sign in link

Gdy `showSignOut=true`, link "Sign in" nie powinien być widoczny — test weryfikuje wzajemne wykluczanie się stanów.

2

DisciplinePicker

`src/components/discipline-picker/discipline-picker.test.tsx`

JEDNOSTKOWY

17 TESTÓW

RENDEROWANIE

T11 renders all three disciplines

Sprawdza, że w DOM są trzy checkboxy: "Crossfit", "Triathlon", "Motorsports". Wyszukiwane przez `role="checkbox"` — przyciski mają `role="checkbox"` i `aria-checked` zamiast być natywnym `<input type="checkbox">`.

T12 renders Disciplines legend

Sprawdza obecność tekstu "Disciplines" — nagłówek sekcji (`<legend>` lub `<span>`) informujący użytkownika, czego dotyczy ta grupa przycisków.

T13 all disciplines start unchecked

Renderuje bez żadnych propsów. Sprawdza, że wszystkie trzy checkboxy mają `aria-checked="false"` — domyślny stan bez wyborów.

T14 renders no hidden inputs when nothing is selected

Używa `container.querySelectorAll('input[type="hidden"]')` i sprawdza, że wynik ma długość 0. Ukryte inputy istnieją tylko dla zaznaczonych dyscyplin — formularz musi wiedzieć co wysłać do serwera.

WARTOŚCI DOMYŚLNE (PROP DEFAULTVALUE)

T15 pre-selects disciplines from defaultValue

Przekazuje `defaultValue={['crossfit', 'triathlon']}`. Sprawdza, że Crossfit i Triathlon mają `aria-checked="true"`, a Motorsports `"false"`. Używane przy edycji istniejącej kampanii.

T16 renders hidden inputs for pre-selected disciplines

Przy dwóch prewybranych dyscyplinach są dokładnie 2 ukryte inputy w DOM od razu po renderowaniu (bez żadnej interakcji).

ZACHOWANIE TOGGLE (KLIKANIE)

T17 clicking a discipline selects it

Kliknięcie Crossfit przez `userEvent.click()` zmienia jego `aria-checked` na `"true"`. Weryfikuje podstawowe działanie toggle.

T18 clicking a selected discipline deselects it

Startuje z `defaultValue={['crossfit']}` (Crossfit zaznaczony). Kliknięcie Crossfit zmienia `aria-checked` z powrotem na `"false"` — toggle działa w obie strony.

T19 selecting one discipline does not affect others

Po kliknięciu Crossfit sprawdza, że Triathlon i Motorsports nadal mają `aria-checked="false"`. Dyscypliny są niezależne — nie ma logiki radio-button.

T20 multiple disciplines can be selected simultaneously

Klika Crossfit i Motorsports (pomijając Triathlon). Sprawdza: Crossfit= `true`, Triathlon= `false`, Motorsports= `true`. Potwierdza, że można zaznaczyć kilka naraz — wielokrotny wybór.

UKRYTE INPUTY (INTEGRACJA Z FORMULARZEM)**T21 adds a hidden input when a discipline is selected**

Po kliknięciu Crossfit pojawia się w DOM element `<input name="disciplines" value="crossfit" type="hidden">`. Ten input jest niezbędny, żeby wartość trafiła do `FormData` przy submit.

T22 renders one hidden input per selected discipline

Klika Crossfit i Triathlon. Sprawdza, że są dokładnie 2 ukryte inputy. Każda dyscyplina ma swój własny input — serwer odczytuje je jako tablicę wartości pod tym samym kluczem.

T23 removes hidden input when discipline is deselected

Startuje z `defaultValue={['crossfit']}`. Po kliknięciu Crossfit (odznaczenie) sprawdza, że input `name="disciplines" value="crossfit"` NIE istnieje w DOM.

T24 uses the name prop for hidden inputs

Renderuje z `name="sports"`. Po kliknięciu Crossfit sprawdza, że input ma `name="sports"` zamiast domyślnego `"disciplines"`. Umożliwia reużycie komponentu z innym kluczem formularza.

STAN BŁĘDU**T25 displays error message when provided**

Przekazanie `error="Select at least one discipline."` renderuje ten tekst w DOM. Błąd pochodzi z serwera (Zod) i jest przekazywany przez formularz nadrzędny.

T26 error message has aria-live="polite"

Element z tekstem błędu ma atrybut `aria-live="polite"`. Powoduje to, że screen readersy ogłoszą pojawienie się błędu gdy skończy się aktualna czytana treść — bez przerywania użytkownikowi.

T27 does not display error element when no error is provided

Renderuje bez prop `error`. Używa `queryByText(/select at least/i)` i sprawdza `.not.toBeInTheDocument()` — brak błędu nie powinien zostawiać pustego kontenera w DOM.

3

Strona logowania

`src/app/(main)/(customer)/login/page.test.tsx`

JEDNOSTKOWY

8 TESTÓW

Setup: `useActionState` jest mockowany przez `vi.mock('react', ...)`. Domyślnie zwraca `[undefined, vi.fn(), false]` — brak stanu, no-op action, nie w toku.

RENDEROWANIE

T28 renders the page heading

Sprawdza, że element z rolą `heading` i tekstem "Sign in to your account" jest w DOM — weryfikuje tytuł strony logowania.

T29 renders email and password fields

Sprawdza obecność inputów przez `getByLabelText('Email address')` i `getByLabelText('Password')` — weryfikuje powiązanie `<label>` z `<input>` przez `htmlFor/id`.

T30 renders the Sign In button

Sprawdza przycisk "Sign In" przez `getByRole('button', { name: 'Sign In' })`.

T31 renders sign up link pointing to /register

Sprawdza link "Sign up" z `href="/register"` — link do strony rejestracji dla nowych użytkowników.

STANY BŁĘDÓW (DANE Z SERWERA)

T32 displays email error from server

Mockuje `useActionState` z `{ errors: { email: ['Email is required.'] } }`. Sprawdza, że tekst "Email is required." jest widoczny. `email` to tablica — komponent renderuje `state.errors.email[0]`.

T33 displays password error from server

Analogicznie do T32 — mockuje `errors.password: ['Password is required.']}` i sprawdza tekst błędu.

T34 displays general error from server

Mockuje `errors.general: 'Invalid login credentials.'` (string, nie tablica). Renderowane bezpośrednio: `{state.errors.general}`. Pojawia się gdy dane logowania są nieprawidłowe.

STAN OCZEKIWANIA (PENDING)

T35 shows Signing in... and disables button while submitting

Mockuje `useActionState` z `pending=true` . Sprawdza, że przycisk ma tekst "Signing in..." (zamiast "Sign In") i jest wyłączony (`disabled`). Zapobiega podwójnemu submitowi.

4

Strona rejestracji

`src/app/(main)/(customer)/register/page.test.tsx`

JEDNOSTKOWY

12 TESTÓW

RENDEROWANIE

T36 renders account type selection buttons

Sprawdza obecność przycisków "Athlete" i "Brand" przez `getByRole('button', { name: ... })`. Są to przyciski toggle do wyboru typu konta.

T37 renders email and password fields

Sprawdza pola email i hasła przez `getByLabelText()`.

T38 renders the Join button

Sprawdza przycisk "Join" — główny przycisk submit formularza rejestracji.

T39 renders sign in link pointing to /login

Sprawdza link "Sign in" z `href="/login"` dla użytkowników, którzy już mają konto.

WYBÓR TYPU KONTA

T40 Athlete button is not pressed by default

Sprawdza, że przycisk "Athlete" ma `aria-pressed="false"` bez żadnej interakcji — domyślnie żaden typ nie jest wybrany.

T41 clicking Athlete marks it as pressed

Po kliknięciu "Athlete": `aria-pressed="true"`. "Brand" nadal `"false"`. Ukryty input `name="account-type"` przechowuje wybraną wartość do przesłania formularzem.

T42 clicking Brand marks it as pressed and deselects Athlete

Klika "Athlete", potem "Brand". Weryfikuje, że "Brand" ma `aria-pressed="true"` a "Athlete" wraca do `"false"`. Zachowanie jak radio-button — tylko jeden może być aktywny.

STANY BŁĘDÓW

T43 displays account type error from server

Mockuje `errors.accountType: ['Select account type.']`. Pojawia się gdy użytkownik nie wybrał "Athlete" ani "Brand" przed wysłaniem formularza.

T44 displays email error from server

Mockuje `errors.email: ['Email is required.']` — renderowane przez `state.errors.email[0]`.

T45 displays password error from server

Mockuje `errors.password: ['Password must be at least 8 characters.']`.

T46 displays general error from server

Mockuje `errors.general: 'User already registered.'` (string). Pojawia się gdy email jest już zajęty w Supabase.

STAN OCZEKIWANIA**T47 shows Joining... and disables button while submitting**

Gdy `pending=true`, przycisk "Join" zmienia tekst na "Joining..." i jest wyłączony — zapobiega wielokrotnej rejestracji podczas oczekiwania na odpowiedź.

5

Formularz onboardingu

```
src/app/(main)/(customer)/onboarding/onboarding-form.test.tsx
```

JEDNOSTKOWY

12 TESTÓW

Komponent OnboardingForm przyjmuje prop `accountType`: `'athlete'` | `'brand'` i renderuje różne pola w zależności od typu konta.

TYP KONTA: ATLETA

T48 renders first name and last name fields

Dla `accountType="athlete"` są pola "First name" i "Last name" — dane osobowe atlety.

T49 does not render brand name field

Atleta NIE ma pola "Brand name" — `queryByText('Brand name')` zwraca `null`.

TYP KONTA: BRAND

T50 renders brand name field

Dla `accountType="brand"` jest pole "Brand name" — nazwa firmy/marki.

T51 does not render first name and last name fields

Brand NIE ma pól imienia i nazwiska — sprawdza, że oba są nieobecne w DOM.

DISCIPLINEPICKER (WSPÓLNY DLA OBU TYPÓW)

T52 renders discipline picker for athlete

Formularz atlety zawiera checkboxy Crossfit, Triathlon i Motorsports — atleta określa swoje dyscypliny podczas onboarding.

T53 renders discipline picker for brand

Formularz brandu również zawiera DisciplinePicker — brand określa, jakie dyscypliny sportu obsługuje (do dopasowywania z atletami).

STANY BŁĘDÓW

T54 displays first name error from server

Mockuje `errors.first_name: ['First name is required.']}`. Pojawia się gdy atleta nie wpisał imienia.

T55 displays last name error from server

Mockuje `errors.last_name: ['Last name is required.']}` . Analogicznie do T54.

T56 displays brand name error from server

Mockuje `errors.brand_name: ['Brand name is required.']}` . Dla formularza brandu.

T57 displays disciplines error from server

Mockuje `errors.disciplines: ['Select at least one discipline.']}` . Błąd przekazywany do `DisciplinePicker` jako prop `error` — renderowany przez jego własny mechanizm błędów (z `aria-live`).

T58 displays general error from server

Mockuje `errors.general: 'Not authenticated.'` — błąd gdy sesja wygasła lub coś poszło nie tak po stronie serwera.

STAN OCZEKIWANIA**T59 shows Saving... and disables button while submitting**

Gdy `pending=true` , przycisk "Continue" zmienia tekst na "Saving..." i jest wyłączony.

6

Formularz kampanii

```
src/app/dashboard/(brand)/campaigns/_components/campaign-form.test.tsx
```

JEDNOSTKOWY

16 TESTÓW

CampaignForm działa w dwóch trybach: **tworzenia** (brak prop campaign) i **edycji** (z prop campaign). Mockuje zarówno createCampaign jak i updateCampaign.

TRYB TWORZENIA (BRAK PROP CAMPAIGN)

T60 renders title and description fields

Pola "Title" i "Description" są w DOM — wyszukiwane przez `getByLabelText()`.

T61 renders discipline picker with all options

Formularz zawiera DisciplinePicker z checkboxami Crossfit, Triathlon i Motorsports.

T62 renders Create Campaign button

Przycisk submit ma tekst "Create Campaign" — sygnalizuje tryb tworzenia nowej kampanii.

T63 title and description inputs start empty

Oba pola mają pustą wartość (`toHaveValue('')`) — nowa kampania startuje z czystym formularzem.

T64 all disciplines start unchecked

Wszystkie trzy checkboxy mają `aria-checked="false"` — brak prewybranych dyscyplin dla nowej kampanii.

TRYB EDYCJI (Z PROP CAMPAIGN)

T65 pre-fills title and description from campaign

Przekazuje `mockCampaign` z tytułem "Summer Sprint" i opisem "A campaign for summer athletes.". Sprawdza, że oba pola mają te wartości przez `toHaveValue()`.

T66 pre-selects disciplines from campaign

`mockCampaign.disciplines = ['crossfit', 'triathlon']`. Crossfit i Triathlon mają `aria-checked="true"`, Motorsports `"false"` — formularz odzwierciedla istniejący stan kampanii.

T67 renders Save Changes button

W trybie edycji przycisk submit ma tekst "Save Changes" zamiast "Create Campaign" — jasny sygnał dla użytkownika, że modyfikuje istniejącą kampanię.

T68 handles campaign with null disciplines

Przekazuje `{ ...mockCampaign, disciplines: null }`. Sprawdza, że wszystkie checkboxy mają `aria-checked="false"` — bezpieczna obsługa kampanii bez przypisanych dyscyplin (edge case z bazy danych).

STANY BŁĘDÓW

T69 displays title error

Mockuje `errors.title: ['Title must be at least 3 characters.']` i sprawdza tekst błędu pod polem tytułu.

T70 title input has aria-invalid when there is an error

Gdy jest błąd tytułu, input ma `aria-invalid="true"` — informuje technologie asystujące, że pole zawiera błędną wartość. Ważne dla dostępności.

T71 displays description error

Mockuje `errors.description: ['Description must be at least 10 characters.']`.

T72 displays disciplines error via DisciplinePicker

Mockuje `errors.disciplines: ['Select at least one discipline.']`. Błąd przekazywany do `DisciplinePicker` — sprawdza, że integracja między formularzem a pickerem działa poprawnie.

T73 displays general error

Mockuje `errors.general: 'Unauthorized.'` — błąd gdy sesja użytkownika wygasła podczas edycji.

STAN OCZEKIWANIA

T74 shows Creating... and disables button in create mode

W trybie tworzenia gdy `pending=true` : przycisk zmienia tekst na "Creating..." i jest wyłączony.

T75 shows Saving... and disables button in edit mode

W trybie edycji gdy `pending=true` : przycisk zmienia tekst na "Saving..." i jest wyłączony.

7

Przycisk usuwania kampanii

```
src/app/dashboard/(brand)/campaigns/_components/delete-button.test.tsx
```

JEDNOSTKOWY

6 TESTÓW

RENDEROWANIE

T76 renders Delete button

Sprawdza, że przycisk z tekstem "Delete" jest w DOM. Komponent `DeleteButton` przyjmuje `id` kampanii jako prop.

T77 renders hidden input with the campaign id

Sprawdza przez `container.querySelector('input[name="id"]')`, że ukryty input ma `value="abc-123"` — ID kampanii przesyłane do server action przy usuwaniu.

T78 button is enabled by default

Sprawdza, że przycisk NIE jest wyłączony w stanie domyślnym — użytkownik może kliknąć Delete.

STAN OCZEKIWANIA

T79 shows Deleting... while pending

Gdy `pending=true`, przycisk zmienia tekst na "Deleting..." — wizualny feedback podczas usuwania.

T80 disables button while pending

Podczas usuwania przycisk jest wyłączony — zapobiega wielokrotnemu kliknięciu Delete na tę samą kampanię.

STAN BŁĘDU

T81 displays error message when deletion fails

Mockuje `{ error: 'Could not delete campaign. Please try again.' }`. Sprawdza, że tekst błędu jest widoczny — informuje użytkownika gdy usunięcie się nie powiodło (np. błąd bazy danych).

8

Rejestracja

tests/e2e/registration.spec.ts

E2E

5 TESTÓW

Każdy test zaczyna od przejścia na /register przez `test.beforeEach`. Testy E2E uderzają w prawdziwy serwer i bazę Supabase.

E1 displays the registration form

Sprawdza widoczność nagłówka "Create your account", przycisków "Athlete" i "Brand" (wybór typu konta), pól email i hasła.

E2 shows error when account type is not selected

Wypełnia email i hasło, ale **nie klika** "Athlete" ani "Brand". Po kliknięciu "Join" serwer zwraca błąd "Select account type." — ukryty input `account-type` jest pusty, Zod go odrzuca.

E3 shows error when password is too short

Wybiera typ "Athlete", wpisuje hasło "123" (3 znaki). Po wysłaniu serwer zwraca "Password must be at least 8 characters." — Zod waliduje minimalną długość hasła.

E4 successful registration as athlete redirects to onboarding

Wybiera "Athlete", wpisuje unikalny email (timestamp) i hasło "password123", klika "Join". Sprawdza redirect na `/onboarding` z nagłówkiem "Complete your profile" i polami "First name", "Last name".

E5 successful registration as brand redirects to onboarding with brand name field

Analogicznie do E4, ale typ "Brand". Po redirectie na `/onboarding` widoczne jest pole "Brand name" zamiast imienia i nazwiska.

9

Logowanie i wylogowanie

`tests/e2e/login.spec.ts`

E2E

7 TESTÓW

Helperzy: `registerUser()` rejestruje nowego użytkownika i czeka na `/onboarding`. `completeOnboarding()` wypełnia onboarding atlety. `signOutUser()` przechodzi na `/dashboard` i klika Sign out.

LOGOWANIE (KAŻDY TEST ZACZYNA OD `PAGE.GOTO('/LOGIN')`)

E6 displays the login form

Sprawdza widoczność nagłówka "Sign in to your account", pól email i hasła, przycisku "Sign In" oraz linku "Sign up".

E7 shows validation errors when fields are empty

Usuwa atrybut `required` ze wszystkich pól przez `page.evaluate()` (żeby przeglądarka nie blokowała submitu). Klika "Sign In" z pustymi polami. Serwer zwraca "Email is required." i "Password is required."

E8 shows error for invalid credentials

Wpisuje nieistniejący email i złe hasło. Supabase zwraca błąd, server action mapuje go na "Invalid login credentials." — sprawdzane przez regex `/invalid login credentials/i`.

E9 successful login redirects to `/onboarding` when onboarding not completed

Rejestruje nowego użytkownika (który ląduje na `/onboarding`), wylogowuje go przez `signOutUser()` (wchodzi na `/dashboard`, klika Sign out), loguje ponownie. Sprawdza URL `/onboarding` — DAL wykrywa brak ukończonego onboarding.

E10 successful login redirects to `/dashboard` when onboarding is completed

Rejestruje, kończy onboarding (John Doe + Crossfit), wylogowuje, loguje ponownie. Sprawdza URL `/dashboard` — profil w Supabase jest ukończony, DAL przekierowuje na dashboard.

WYLOGOWANIE

E11 dashboard header shows sign out button when logged in

Rejestruje użytkownika, wchodzi na `/dashboard`. Sprawdza widoczność przycisku "Sign out" w headerze — weryfikuje, że zalogowany użytkownik ma możliwość wylogowania.

E12 sign out redirects to homepage

Rejestruje użytkownika, klika Sign out przez `signOutUser()` . Sprawdza URL / — po wylogowaniu użytkownik wraca na stronę główną.

10

Onboarding

tests/e2e/onboarding.spec.ts

E2E

8 TESTÓW

ONBOARDING — ATLETA (BEFOREEACH: REJESTRACJA JAKO ATHLETE)

E13 displays form with first name and last name fields

Sprawdza widoczność nagłówka "Complete your profile" i pól "First name", "Last name" dla atlety.

E14 shows server validation errors when fields are empty

Usuwa `required` ze wszystkich pól, klika "Continue". Serwer waliduje przez Zod i zwraca trzy błędy jednocześnie: "First name is required.", "Last name is required.", "Select at least one discipline."

E15 redirects to home on successful submission

Wypełnia "John", "Doe", zaznacza Crossfit, klika "Continue". Supabase aktualizuje profil. Server action przekierowuje na `/dashboard`.

ONBOARDING — JUŻ UKOŃCZONY

E16 redirects to home if onboarding already done

Rejestruje i kończy onboarding. Następnie próbuje wejść ponownie na `/onboarding` przez `page.goto()`. DAL wykrywa ukończony onboarding i przekierowuje na `/dashboard` — ochrona przed ponownym wypełnianiem formularza.

ONBOARDING — BRAND (BEFOREEACH: REJESTRACJA JAKO BRAND)

E17 displays form with brand name field

Sprawdza widoczność pola "Brand name" dla brandu — serwer renderuje inny formularz w zależności od typu konta przechowywanego w profilu Supabase.

E18 shows server validation error when field is empty

Usuwa `required`, klika "Continue". Serwer zwraca "Brand name is required." i "Select at least one discipline."

E19 redirects to home on successful submission

Wypełnia "Nike", zaznacza Crossfit, klika "Continue". Redirect na `/dashboard`.

11

Dashboard — kontrola dostępu

tests/e2e/dashboard.spec.ts

E2E

4 TESTY

KONTROLA DOSTĘPU DO DASHBOARDU

E20 unauthenticated user is redirected to /login

Wchodzi na `/dashboard` bez logowania. Middleware/DAL wykrywa brak sesji i przekierowuje na `/login`.

E21 authenticated user without completed onboarding is redirected to /onboarding

Rejestruje użytkownika (który łąduje na `/onboarding`), następnie przez `page.goto('/dashboard')` próbuje wejść na dashboard. DAL sprawdza profil — brak `first_name` (lub `brand_name`), redirect na `/onboarding`.

E22 authenticated user with completed onboarding can access dashboard

Rejestruje atletę, kończy onboarding (John Doe + Crossfit). Sprawdza URL `/dashboard` — ukończony profil pozwala na dostęp.

TRASY TYLKO DLA BRANDÓW

E23 athlete account is redirected from /dashboard/campaigns to /dashboard

Rejestruje atletę, kończy onboarding. Próbuje wejść na `/dashboard/campaigns` przez `page.goto()`. Layout brandu (`brand`) sprawdza typ konta i przekierowuje atletę na `/dashboard` — trasy kampanii są tylko dla brandów.

12

Kampanie

`tests/e2e/campaigns.spec.ts`

E2E

19 TESTÓW

Helperzy: `registerBrand()` rejestruje brand i kończy onboarding (Brand name + Crossfit). `createCampaign()` tworzy kampanię przez formularz i czeka na redirect do listy.

LISTA KAMPANII (BEFOREEACH: REGISTERBRAND + GOTO '/DASHBOARD/CAMPAIGNS')

E24 displays empty state when no campaigns exist

Nowy brand nie ma żadnych kampanii. Sprawdza tekst "No campaigns yet." i link "Create your first campaign →" — stan pusty z CTA.

E25 displays New Campaign button

Sprawdza widoczność linku "New Campaign" prowadzącego do formularza tworzenia kampanii.

E26 shows created campaign in the list

Tworzy kampanię "My First Campaign" przez `createCampaign()`. Po redirect na listę sprawdza widoczność tytułu kampanii.

E27 new campaign has draft status badge

Tworzy kampanię "Draft Campaign". Sprawdza `getByText('draft', { exact: true })` — szuka dokładnego dopasowania tekstu "draft" (badge statusu). Bez `{ exact: true }` Playwright znalazłby też tytuł "Draft Campaign" i opis, co spowodowałoby błąd strict mode.

TWORZENIE KAMPANII (BEFOREEACH: REGISTERBRAND + GOTO '/DASHBOARD/CAMPAIGNS/NEW')

E28 displays the creation form

Sprawdza widoczność pól Title, Description, checkboxów Crossfit/Triathlon/Motorsports i przycisku "Create Campaign".

E29 shows validation errors when fields are empty

Usuwa `required`, klika "Create Campaign". Serwer zwraca trzy błędy: "Title must be at least 3 characters.", "Description must be at least 10 characters.", "Select at least one discipline."

E30 successful creation redirects to campaigns list and shows the campaign

Wypełnia tytuł "Launch Campaign", opis, zaznacza Triathlon, klika "Create Campaign". Sprawdza redirect na `/dashboard/campaigns` i widoczność tytułu kampanii na liście.

E31 Back to Campaigns link navigates to list

Klika link "Back to Campaigns" przez `getByRole('link', { name: /back to campaigns/i })`. Sprawdza redirect na `/dashboard/campaigns`.

PODGLĄD KAMPANII (BEFOREEACH: REGISTERBRAND + CREATECAMPAIGN + KLIK PREVIEW)**E32 displays campaign title and description**

Na stronie podglądu (`/dashboard/campaigns/[id]`) sprawdza nagłówek "Preview Campaign" i tekst opisu "Full campaign description here."

E33 displays campaign disciplines

Sprawdza, że tekst "Triathlon" jest widoczny na stronie podglądu — dyscyplina wyświetlana jako badge.

E34 displays Edit and Delete actions

Sprawdza widoczność linku "Edit" (prowadzącego do formularza edycji) i przycisku "Delete" (otwierającego dialog potwierdzenia).

E35 Back to Campaigns link navigates to list

Klika "Back to Campaigns" na stronie podglądu. Sprawdza redirect na `/dashboard/campaigns`.

EDYCJA KAMPANII (BEFOREEACH: REGISTERBRAND + CREATECAMPAIGN + PREVIEW + EDIT)**E36 pre-fills form with existing campaign data**

Formularz edycji ma wypełniony tytuł "Original Title", opis "Original description text here." i zaznaczoną dyscyplinę Crossfit (`aria-checked="true"`).

E37 successful update redirects to campaigns list and shows updated title

Czyści pole tytułu (`clear()`), wpisuje "Updated Title", klika "Save Changes". Sprawdza redirect na listę i widoczność nowego tytułu — aktualizacja zapisywana w Supabase.

E38 shows validation error when title is too short

Usuwa `required`, wpisuje "ab" (2 znaki — za krótki). Klika "Save Changes". Serwer zwraca "Title must be at least 3 characters." — weryfikuje walidację przy edycji.

E39 can change selected disciplines and changes are saved

Odnacza Crossfit, zaznacza Triathlon, klika "Save Changes". Czeki na `/dashboard/campaigns`, klika Preview. Sprawdza "Triathlon" jest widoczny i "Crossfit" NIE jest widoczny — zmiana dyscyplin trwale zapisana.

WIELE DYSCYPLIN

E40 all selected disciplines are saved and displayed in preview

Tworzy kampanię z zaznaczonymi Crossfit, Triathlon i Motorsports (wszystkie trzy). Po wejściu w Preview sprawdza, że wszystkie trzy są widoczne — weryfikuje poprawne przesyłanie wielu wartości przez FormData i zapisywanie w bazie.

USUWANIE KAMPANII**E41 deletes campaign and removes it from the list**

Tworzy kampanię "To Be Deleted", wchodzi w Preview. Rejestruje handler dialogu `page.once('dialog', dialog => dialog.accept())` przed kliknięciem "Delete". Po akceptacji `window.confirm` : redirect na listę, kampania NIE jest widoczna.

E42 cancelling deletion keeps the campaign

Ten sam scenariusz, ale `dialog.dismiss()` — anulowanie potwierdzenia. Kampania pozostaje, nagłówek "Keep This Campaign" jest nadal widoczny — weryfikuje, że anulowanie nie wywołuje usunięcia.

Zestawienie testów według pliku

#	PLIK	TYP	TESTÓW	TESTOWANE OBSZARY
1	header.test.tsx	UNIT	10	Renderowanie, linki nav, sign in/out, logo
2	discipline-picker.test.tsx	UNIT	17	Renderowanie, defaultValue, toggle, hidden inputs, błędy
3	login/page.test.tsx	UNIT	8	Renderowanie, błędy email/hasło/general, pending
4	register/page.test.tsx	UNIT	12	Renderowanie, wybór typu konta, błędy serwera, pending
5	onboarding-form.test.tsx	UNIT	12	Typy konta, DisciplinePicker, błędy serwera, pending
6	campaign-form.test.tsx	UNIT	16	Tryb tworzenia i edycji, null disciplines, błędy, aria-invalid
7	delete-button.test.tsx	UNIT	6	Renderowanie, hidden id, pending, błąd usuwania
8	registration.spec.ts	E2E	5	Formularz, walidacja, redirect po rejestracji (athlete/brand)
9	login.spec.ts	E2E	7	Formularz, walidacja, złe dane, redirect po logowaniu, wylogowanie
10	onboarding.spec.ts	E2E	8	Athlete/brand formularz, walidacja, redirect, ochrona przed powrotem
11	dashboard.spec.ts	E2E	4	Kontrola dostępu, middleware auth, trasy brand-only
12	campaigns.spec.ts	E2E	19	Lista, tworzenie, podgląd, edycja, wiele dyscyplin, usuwanie
Łącznie			124	81 jednostkowych + 43 E2E